

DC Motor, Locked Rotor Test, data processing

Packages

```
In [ ]: import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
import numpy as np
import seaborn as sns
import os
import json
```

Configuration data.

```
In [ ]: # Folder name
CSV_folder_name = "CSV_files"
figure_folder_name = "figures"
json_folder_name = "json"

# Motor data file name and default values
motor_file = 'motor_data.json'
default_motor_info = {
    'R_a': 1,
    'K_V': 1,
    'K_m': 1
}

# Option to save data and figures ( True = Save data and figures, False = Don't save)
SAVE = False
```

Load data set

```
In [ ]: # Load data from CSV file
file_name = "locked_rotor2.csv" # CSV name
file_path = os.path.join(CSV_folder_name, file_name) # Generate file path
raw_df = pd.read_csv(file_path) # Read CSV file.
```

Calculating the internal resistance of the motor.

Equation for calculating internal resistance:

$$R_a = \frac{V - V_{shunt}}{i_a} \quad (1)$$

Where:

- R_a = Internal resistance of the motor in ohms. $[\Omega]$.
- V = Applied voltage in volts $[V]$.
- V_{shunt} = Voltage drop from current measuring resistor in volts $[V]$.

- i_a = Current passing through motor in ampere [A].

```
In [ ]: # Select example sample from data set
sample = 5
voltage_step = 1 # Voltage step-up per sample, 1 = 1V, 2V, 3V , 4V etc...

# Create new dataframe for rotor data, containing rotor position and time
voltage_data = pd.concat([raw_df.iloc[:, (5*sample + 1)], raw_df.iloc[:, (5*sample
voltage_data.columns = ['V_bus', 'V_shunt', "Current", "Time"]

# Change V_shunt from mV to V and Current from mA to A
voltage_data['V_shunt'] = voltage_data['V_shunt'] / 1000
voltage_data["Current"] = voltage_data["Current"] / 1000

# Calculate armature resistance
voltage_data["Armature Resistance"] = ( voltage_step*sample - voltage_data['V_shunt
armature_resistance = abs(voltage_data['Armature Resistance'].mean()) # Calcualte n

# Print estimated resistance
print("Estimated armature resistance for sample {:.0f}, with voltage step of {:.0f})
```

Estimated armature resistance for sample 5, with voltage step of 1V: 6.13Ω.

Apply above calculations to the entire data set

```
In [ ]: # Voltage step-up per sample, 1 = 1V, 2V, 3V , 4V etc...
voltage_step = 1

# Create an empty DataFrame for motor constant
resistance_array = np.array([])

# Iterating through every fifth column
for sample in range(0, len(raw_df.columns), 5):
    ## Create new dataframe for voltage data
    voltage_data = pd.concat([raw_df.iloc[:, (sample + 1)], raw_df.iloc[:, (sample
    voltage_data.columns = ['V_bus', 'V_shunt', "Current", "Time"]

    ## Change V_shunt from mV to V and Current from mA to A
    voltage_data['V_shunt'] = voltage_data['V_shunt'] / 1000
    voltage_data["Current"] = voltage_data["Current"] / 1000

    ## Calculate applied voltage for dataset
    voltage_data["V_applied"] = voltage_step*(sample/5)

    ## Calculate armature resistance.
    voltage_data["Armature Resistance"] = ( voltage_data["V_applied"] - voltage_dat

    ## Replace inf with NaN
    voltage_data.replace([np.inf, -np.inf], np.nan, inplace=True)

    ## Remove rows with NaN values
    voltage_data = voltage_data.dropna()

    ## Store resistance estimates in array
    resistance_array = np.append(resistance_array, abs(voltage_data['Armature Resis
```

```
# Calculate mean armature resistance
armature_resistance = np.mean(resistance_array)

# Calculate mean, mode and median
armature_resistance = np.mean(resistance_array)

# Print values
print("Estimated mean armature resistance: {:.2f}Ω.".format(armature_resistance))
print("")

# Save data
if SAVE:
    ## Generate file path
    file_path = os.path.join(json_folder_name, motor_file)

    ## Check if the file exists
    if os.path.exists(file_path):
        ### If file exists, read the values from it
        with open(file_path, 'r') as json_file:
            motor_data = json.load(json_file)
    else:
        ### If file doesn't exist, use default values
        motor_data = default_motor_info

    ## Print old motor data
    print("Old motor data:")
    print("R_a = {:.2f}".format(motor_data['R_a']))
    print("K_V = {:.4f}".format(motor_data['K_V']))
    print("K_m = {:.4f}".format(motor_data['K_m']))
    print("")

    ## Save resistance
    motor_data['R_a'] = armature_resistance

    ## Save the updated dictionary to the JSON file
    with open(file_path, 'w') as json_file:
        json.dump(motor_data, json_file)

    ## Print new motor data
    print("New motor data:")
    print("R_a = {:.2f}".format(motor_data['R_a']))
    print("K_V = {:.4f}".format(motor_data['K_V']))
    print("K_m = {:.4f}".format(motor_data['K_m']))
    print("")
```

Estimated mean armature resistance: 5.80Ω.